Hafta 6 — Model Doğrulama

> **Ders:** Web Programlama · **Hafta:** 6 / 13 ders haftası · **Süre:** 3 × 50 dakika **Ön koşul:** Hafta 4 (Model Binding, `ModelState`, PRG) ve Hafta 5 (`asp-for`, `data-val-*`) **Bu dosya sınav kapsamındadır.** Daha derin konular `H06-detay.md` dosyasındadır ve **sınav kapsamı dışıdır.**

🎯 Bu Haftanın Hedefleri

Bu haftanın sonunda:

1. Bir isteğin **Model Binding → doğrulama → action** sırasını ve `ModelState`'in bu sırada neyi taşıdığını **anlatabileceksiniz**.
2. Attribute yazmadan gelen **örtük** doğrulamanın kaynaklarını (tip, nullable işareti, bağlama) **gösterebileceksiniz**.
3. Verilen bir modele `[Required]`, `[StringLength]`, `[Range]`, `[EmailAddress]`, `[Compare]`, `[RegularExpression]` ile **kural yazabileceksiniz**.
4. `ErrorMessage` ve `{0}`, `{1}`, `{2}` yer tutucularıyla **Türkçe mesaj** verebileceksiniz.
5. `ModelState.IsValid` ile **kanonik POST action'ını** yazabilecek ve geçersiz isteğin neden **200** döndüğünü açıklayabileceksiniz.
6. `asp-validation-for` ve `asp-validation-summary` ile hataları **gösterebilecek**, `AddModelError` ile iş kuralı hatası **ekleyebileceksiniz**.
7. İstemci doğrulamasını açabilecek ve **neden güvenlik olmadığını** kanıtlayabileceksiniz.
8. Kendi `ValidationAttribute`'ünüzü ve `IValidatableObject` kuralınızı **yazabileceksiniz**.

🧩 **Haftanın ilkesi — Güvenme, doğrula; ve doğrulamayı denetlediğin tarafta yap**

> Tarayıcıdaki doğrulama kullanıcıya yardım eder, ama tarayıcı **sizin** değildir. Kural modelde yazılır ve **sunucuda** çalışır; istemcideki kopyası yalnızca aynı kuralın erken uyarısıdır.

---

1. Doğrulama Nedir ve Nerede Çalışır?

Doğrulama, istemciden gelen verinin uygulamanın kabul ettiği kurallara uyup uymadığını **işlemden önce** denetlemektir. Hafta 4'ün ilkesi buradan çıkar: istemciden gelen hiçbir veri güvenilir değildir — öyleyse her veri bir kapıdan geçmelidir.

Bu haftanın iki ucu önceki haftalarda açık kaldı. Hafta 4'te `ModelState`'i tanıdık: bağlama başarısız olunca hata fırlatılmıyor, durum oraya yazılıyordu. Hafta 5'te `data-val-*` attribute'lerinin HTML'e kendiliğinden yazıldığını gördük: onları okuyan hiçbir şey yoktu. Bu hafta ikisini birleştiriyoruz: kuralları **modele** yazacağız; çatı onları sunucuda çalıştırıp sonucu `ModelState`'e koyacak, aynı kuralları `data-val-*` olarak tarayıcıya da taşıyacak.

1.1. Üç soru, üç yer

Bir form gönderildiğinde sunucu üç ayrı soru sorar ve her birini ayrı bir yer cevaplar:

Soru	Örnek	Cevaplayan
Bu metin bu **tipe** çevrilebilir mi?	`"abc"` bir `decimal` olabilir mi?	**Model Binding** (Hafta 4)
Değer **kurala** uyuyor mu?	Başlık boş mu, fiyat 1–10000 arasında mı?	**Doğrulama attribute'leri** (bölüm 2)
İşlem **iş kuralına** uyuyor mu?	Bu başlıkta kitap zaten var mı?	**Controller** ya da servis (bölüm 3.4)

Üçünün cevabı **aynı yere** yazılır: `ModelState`.

1.2. Akış: Model Binding → doğrulama → action

```
POST /Kitap/EkleTH
   │
   ├─ 1. Model Binding    form değerlerini KitapEkleVM'ye çevirir; çeviremediğini ModelState'e yazar
   ├─ 2. Doğrulama         attribute'leri ve IValidatableObject'i çalıştırır; hataları ModelState'e ya
   └─ 3. Action            ilk satırı çalıştığında iki iş de BİTMİŞTİR
```

Action'ın içinde doğrulamayı **siz çağırmazsınız**; çatı çağırmış ve sonucu `ModelState`'e bırakmıştır. `ModelState` bir sözlüktür: her anahtar bir alandır. `/Dogrula` sayfası onu tablo hâlinde basar; boş gönderilen formun ölçülen hâli:

Anahtar	Gelen ham metin	Durum	Hata
`Ad`	`""`	Invalid	The Ad field is required.
`Eposta`	`""`	Invalid	The E-posta field is required.
`Yas`	`""`	Invalid	The value '' is invalid.
`Telefon`	`""`	Valid	

Her giriş dört şey taşır:

Parça	Ne	Örnekte
Anahtar	Alanın adı; iç içe nesnede noktalı (`Adres.Sehir`, Hafta 5)	`Yas`
Ham metin (`AttemptedValue`)	İstemcinin gönderdiği metin, çevrilmeden önce	`""`
Durum (`ValidationState`)	`Valid`, `Invalid`, `Unvalidated` ya da `Skipped`	`Invalid`
Hatalar (`Errors`)	O alana yazılmış bütün mesajlar	The value '' is invalid.

Ham metnin ayrıca tutulması bir tasarım kararıdır: değer modele **çevrilemediğinde** bile kullanıcının yazdığı kaybolmaz ve forma geri yazılabilir (bölüm 3.1).

`ModelState.IsValid`, hiçbir anahtarda hata yoksa `true`'dur. Geçersiz istek bir hata sayfası değildir: action formu hatalarla geri gösterir ve yanıt **200** döner. Hafta 5'te token'sız POST'un **400** aldığını görmüştük — o istek action'a hiç ulaşmıyordu; burada istek ulaşır ve action ne yapacağına kendisi karar verir.

1.3. Örtük doğrulama — attribute yazmadan gelen hatalar

`/Dogrula/Ortuk` sayfasındaki modelde **hiçbir** doğrulama attribute'ü yok. Yine de:

Özellik	Boş gönderilince	`abc` gönderilince
`int Sayi`	**The value '' is invalid.**	**The value 'abc' is not valid for Sayi.**
`int? SayiNullable`	geçerli — `null`	**The value 'abc' is not valid...**
`string Metin`	**The Metin field is required.**	geçerli
`string? MetinNullable`	geçerli — `null`	geçerli

Üç kaynak var:

- **Bağlama:** `int` boş metni ya da `abc`'yi taşıyamaz; hata Model Binding'den gelir.
- **Nullable işareti:** `string` (soru işaretsiz) "boş olamaz" demektir; çatı onu `[Required]` yazılmış gibi denetler. `string?` için böyle bir denetim yoktur.
- **"Yok" ile "boş" farkı:** Aynı sayfanın ikinci formu bu alanları **hiç göndermez**. Sonuç: hiçbir hata yok, `Sayi` = `0`. Gelmeyen alan için `ModelState`'te anahtar bile açılmaz. Boş gönderilen alan ise denetlenir.

Bu mesajların **İngilizce** kalması bir ipucudur: onları sizin attribute'ünüz değil, Model Binding yazar. Kendi mesajınızı verdiğiniz kurallar Türkçe görünür (bölüm 2.2); İngilizce bir mesaj gördüğünüzde hatanın **bağlamada** doğduğunu bilirsiniz.

1.4. Üç katman

Aynı kural üç yerde çalışabilir; yalnızca biri sizin denetiminizdedir:

Katman	Nerede	Kim yazar	Atlatılabilir mi?
Tarayıcının yerleşik denetimi	`type="email"`, `maxlength="100"`	`asp-for` attribute'lerden üretir	**Evet**
jQuery Validation	`data-val-*` attribute'lerini okur	`_ValidationScriptsPartial`	**Evet**
Sunucu	`ModelState`	çatı, her istekte	**Hayır**

İlk katman çoğu zaman fark edilmez. `asp-for`, `[EmailAddress]` için `type="email"`, `[Url]` için `type="url"`, `int` için `type="number"` ve `[StringLength(100)]` için `maxlength="100"` üretir; tarayıcı bunları **kendisi** denetler. `/Dogrula` formunda e-postaya `xyz` yazıp düğmeye basıldığında istek hiç gönderilmedi ve tarayıcının kendi uyarısı çıktı — sayfada jQuery Validation yüklü değilken. `[Required]` ise HTML'e `required` attribute'ü **eklemez**; yalnızca `data-val-required` yazar. "Sunucuya neden gitmedi?" sorusunun cevabı bu yüzden bazen jQuery'de değil, tarayıcının kendisindedir.

İlk ikisi kullanıcıya **hızlı geri bildirim** verir; güvenliği yalnızca üçüncüsü sağlar (bölüm 4.3).

---

2. DataAnnotations — Kuralı Modelde Yazmak

Kurallar `System.ComponentModel.DataAnnotations` ad alanındaki attribute'lerle, **ViewModel'in özelliklerine** yazılır. Görünümde değil, modelde: `/Kitap/Ekle` (Hafta 4, elle yazılmış) ve `/Kitap/EkleTH` (Hafta 5, Tag Helper'lı) aynı `KitapEkleVM`'yi kullandığı için **ikisi de** aynı kurallara tabidir.

2.1. Hazır attribute'ler ve varsayılan mesajları

`/Dogrula` sayfasında ölçülen varsayılan mesajlar:

Attribute	Kural	Varsayılan mesaj
`[Required]`	Boş olamaz	The Ad field is required.
`[StringLength(30, MinimumLength = 2)]`	2–30 karakter	The field Ad must be a string with a minimum length of 2 and a maximum length of 30.
`[Range(18, 99)]`	18–99 arası	The field Yaş must be between 18 and 99.
`[EmailAddress]`	E-posta biçimi	The E-posta field is not a valid e-mail address.
`[Compare(nameof(Parola))]`	Başka bir alanla aynı	'Parola (tekrar)' and 'Parola' do not match.
`[RegularExpression(@"^05\d{9}$")]`	Kalıba uyar	The field Cep telefonu must match the regular expression '^05\d{9}$'.
`[Url]`	Tam adres	The Web sitesi field is not a valid fully-qualified http, https, or ftp URL.

Mesajlardaki alan adları (`Yaş`, `Cep telefonu`) `[Display(Name = "…")]`'dan gelir. `[MaxLength]` ve `[MinLength]` de vardır; metin için `[StringLength]` yaygındır, çünkü alt ve üst sınırı tek attribute'te söyler.

Her attribute yalnızca kendi kuralına bakar. Aynı boş formda `Ad` için yalnızca `[Required]`'ın mesajı geldi, `[StringLength]`'inki gelmedi; `Telefon` ve `Web sitesi` boşken **geçerli** sayıldı. `[StringLength]`, `[EmailAddress]`, `[RegularExpression]`, `[Url]` ve `[Range]` boş (`null`) değeri geçerli kabul eder: boşluğu denetlemek yalnızca `[Required]`'ın işidir. Bu yüzden zorunlu olmayan bir telefon alanına yalnızca `[RegularExpression]` yazarsınız: boş bırakılabilir, ama doldurulursa kalıba uymalıdır. Aynı ilke kendi attribute'ünüzü yazarken de geçerlidir (bölüm 5.2).

2.2. `ErrorMessage` ve yer tutucular

Varsayılan mesajlar İngilizcedir ve `[RegularExpression]`'ınki kullanıcıya kalıbın kendisini gösterir. Bu yüzden her kurala kendi mesajınızı verin:

```
[Required(ErrorMessage = "{0} boş bırakılamaz.")]
[StringLength(100, MinimumLength = 2, ErrorMessage = "{0} {2} ile {1} karakter arasında olmalıdır.
[Display(Name = "Başlık")]
public string Baslik { get; set; } = "";
```

Yer tutucu	Ne ile dolar	Örnekte
`{0}`	Alanın görünen adı (`[Display]`)	Başlık

`{1}`	İlk parametre (üst sınır, `[Range]`'de alt sınır)	100
`{2}`	İkinci parametre	2

Ölçülen sonuç: `Başlık boş bırakılamaz.` Dikkat: `[StringLength]`'te `{1}` **üst**, `{2}` **alt** sınırdır; `[Range]`'de sıra ise `{1}` alt, `{2}` üst.

2.3. Değer tiplerinde `[Required]`: nullable olmadan çalışmaz

`decimal Fiyat` alanını boş gönderin. `[Required(ErrorMessage = "…")]` yazmış olsanız bile göreceğiniz mesaj **sizin değil**:

Tanım	Boş gönderilince ölçülen
`[Range(1.0, 10000.0)] decimal Fiyat`	**The value '' is invalid.** — bağlamada düştü
`[Required(…)] [Range(…)] decimal? Fiyat`	**Fiyat (TL) boş bırakılamaz.**

Sebep bölüm 1.2'deki sıradır: `decimal` boş metni taşıyamaz, Model Binding düşer ve sizin kuralınız hiç çalışmaz. `decimal?` ise boşu `null` olarak taşır; `[Required]` `null`'u yakalar. Microsoft'un belgesi de aynı şeyi söyler: sunucuda `[Required]`, değer `null` ise "eksik" sayar; **nullable olmayan bir alan her zaman geçerlidir.** Kural:

> **Değer tipinde (`int`, `decimal`, `DateTime`) `[Required]` ancak tip nullable ise anlamlıdır.**

"— seçiniz —" seçenekli bir `<select>` için de aynısı geçerlidir: `int? KategoriId` + `[Required(ErrorMessage = "Bir kategori seçin.")]`. Bu yüzden `KitapEkleVM`'de `Fiyat` ve `KategoriId` bu hafta nullable oldu (**TODO 6.2**).

2.4. `[DataType]` doğrulamaz

`/Dogrula/Ortuk` sayfasındaki iki e-posta alanına `xyz` yazıldığında:

Tanım	Sunucuda
`[DataType(DataType.EmailAddress)]`	**geçerli**
`[EmailAddress]`	The EpostaAttribute field is not a valid e-mail address.

`[DataType]` alanın **anlamını** söyler — Hafta 5'te gördüğümüz gibi `type="email"` üretir — ama sunucuda hiçbir şeyi denetlemez. `[Display]` gibi o da bir doğrulama attribute'ü değildir.

2.5. Sınavın klasik sorusu: modele kural yazmak

"Üye kayıt formu için ad (zorunlu, 2–30 karakter), e-posta (zorunlu, geçerli biçim), yaş (18–99), parola (zorunlu, en az 6) ve parola tekrarı (parolayla aynı) kurallarını yazın."

```
public class UyeVM
{
    [Required(ErrorMessage = "{0} zorunludur.")]
    [StringLength(30, MinimumLength = 2, ErrorMessage = "{0} {2}–{1} karakter olmalıdır.")]
    [Display(Name = "Ad")]
    public string? Ad { get; set; }

    [Required(ErrorMessage = "{0} zorunludur.")]
    [EmailAddress(ErrorMessage = "Geçerli bir e-posta yazın.")]
    [Display(Name = "E-posta")]
    public string? Eposta { get; set; }

    [Required(ErrorMessage = "{0} zorunludur.")]
    [Range(18, 99, ErrorMessage = "{0} {1} ile {2} arasında olmalıdır.")]
    [Display(Name = "Yaş")]
    public int? Yas { get; set; }

    [Required(ErrorMessage = "{0} zorunludur.")]
    [StringLength(100, MinimumLength = 6, ErrorMessage = "{0} en az {2} karakter olmalıdır.")]
    [DataType(DataType.Password)]
    public string? Parola { get; set; }

    [Compare(nameof(Parola), ErrorMessage = "Parolalar aynı değil.")]
    [DataType(DataType.Password)]
    public string? ParolaTekrar { get; set; }
}
```

Puan kaybettiren üç yer: `Yas`'ı `int` bırakıp `[Required]` yazmak (bölüm 2.3); `[Compare]`'e özellik adını metin olarak yazmak (`nameof` derleyiciyle denetlenir); `[DataType]`'ı doğrulama sanmak (bölüm 2.4).

2.6. Kural nereye yazılır: ViewModel mi, entity mi?

Hafta 4'te formun alabileceği alanları açıkça yazmak için ViewModel kurmuştuk (`KitapEkleVM`); entity (`Kitap`) action parametresi olmuyordu. Doğrulama kuralları da **formun** kurallarıdır ve ViewModel'e yazılır:

	ViewModel'deki kural	**Entity'deki kural**
Ne anlatır	**Bu formun** kabul ettiği veri	Verinin **her yerde** taşıması gereken özellik
Örnek	Kayıt formunda "Parola (tekrar)" zorunlu	Başlık en fazla 100 karakter
Kim kullanır	Model Binding ve doğrulama	Hafta 8'de EF Core da: `[StringLength(100)]` sütun boyutunu belirler

Aynı veri iki formda farklı kurallara tabi olabilir: kayıt formunda parola zorunludur, profil düzenleme formunda değildir. Bu yüzden form kuralları formun ViewModel'inde durur. Bir entity'ye yazılan `[Required]`, o entity'nin bağlandığı **her** formda geçerli olur — Hata Köşesi'ndeki "`IsValid` hep `false`" çoğu zaman buradan doğar.

---

3. `ModelState` ve Sunucu Tarafı Doğrulama

3.1. Kanonik action kalıbı

Bu kalıp Hafta 4'ten beri `KitapController`'da duruyor; bu hafta içi doldu:

```
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult EkleTH(KitapEkleVM girdi)
{
    if (!ModelState.IsValid)
        return View(girdi);                 // 200 — aynı form, hatalarla

    KitapDeposu.Ekle(/* … */);
    return RedirectToAction(nameof(Index)); // 302 — PRG (Hafta 4)
}
```

Durum	Yanıt	Ölçülen zincir
Geçersiz	Form hatalarla geri gelir	`200 POST /Kitap/EkleTH`
Geçerli	Kaydedilir, listeye yönlendirilir	`302 POST` → `200 GET /Kitap`

`View(girdi)`'deki `girdi` önemlidir: kullanıcının yazdıkları kaybolmaz. Daha da fazlası, `asp-for` kutuya modelin değerini değil kullanıcının **gönderdiği ham metni** geri yazar: `Fiyat` alanına `abc` yazılırsa kutu yine `abc` ile gelir (ölçüldü) — kullanıcı neyi yanlış yazdığını görür.

3.2. `IsValid` ne zaman `false` olur?

Kaynak	Örnek
Bağlama hatası	`Fiyat` = `abc` → The value 'abc' is not valid for Fiyat (TL).
Doğrulama attribute'ü	Başlık boş → Başlık boş bırakılamaz.
Örtük `[Required]`	nullable olmayan `string` boş geldi
`IValidatableObject`	başlık ile yazar aynı (bölüm 5.3)
`AddModelError`	aynı başlıkta kitap var (bölüm 3.4)

Bir gönderimde birden çok kaynak birlikte konuşabilir. Kitap formuna başlık `orhan pamuk`, yazar `orhan pamuk`, fiyat `0`, kategori boş gönderildiğinde ölçülen:

Alan	Hata	Kaynak
Başlık	Başlık büyük harfle başlamalıdır.	kendi attribute'ümüz
Fiyat	Fiyat (TL) 1 ile 10000 arasında olmalıdır.	`[Range]`
Kategori	Bir kategori seçin.	`[Required]`

Başlık ile yazar aynı olduğu hâlde "aynı olamaz" hatası **yok** — sebebi bölüm 5.3'te.

Bir istisna Hata Köşesi'ndedir: formda **hiç olmayan** bir alan da `IsValid`'i `false` yapabilir.

3.3. Hataları göstermek: `asp-validation-for` ve `asp-validation-summary`

Hafta 4'teki elle yazılmış formda hataları `ModelState` üzerinde döngüyle listeliyorduk. İki Tag Helper bu işi yapar (**TODO 6.3**):

```
<div asp-validation-summary="ModelOnly"></div>

<input asp-for="Baslik" />
<span asp-validation-for="Baslik"></span>
```

Hata varken üretilen HTML:

```
<input class="input-validation-error" … id="Baslik" name="Baslik" value="" />
<span class="field-validation-error" data-valmsg-for="Baslik" data-valmsg-replace="true">Başlık bo
```

Sınıf adlarını (`input-validation-error`, `field-validation-error`) **çatı** yazar; kırmızı çerçeve `site.css`'teki bu sınıflardan gelir.

Action hataları görünüme ayrıca göndermedi: yalnızca `View(girdi)` dedi. Tag Helper'lar `ModelState`'e görünümün bağlamından (`ViewContext`) erişir. `asp-for` bir alanı çizerken o alanın anahtarına bakar: hata varsa sınıfı ekler, ham metin varsa onu yazar; `asp-validation-for` aynı anahtarın mesajını basar. `ModelState` action ile görünüm arasında **kendiliğinden** taşınır — Hafta 4'te `ViewData` için gördüğümüz gibi, tek bir istek boyunca.

Özet türü	Ne listeler	Hata yokken
`All`	Bütün hatalar	Gizli, boş bir `<ul>` üretir
`ModelOnly`	Yalnızca **bir alana bağlı olmayan** hatalar	**Hiçbir şey** üretmez

Alanların altında zaten `asp-validation-for` varsa `ModelOnly` kullanın; yoksa aynı hata iki kez görünür.

3.4. `AddModelError` — iş kuralı hatası eklemek

"Bu başlıkta kitap zaten var mı?" sorusunu bir attribute cevaplayamaz: veri deposuna bakmak gerekir. Kontrol action'da yapılır ve sonuç **aynı yere** yazılır (**TODO 6.4**):

```
if (KitapDeposu.Hepsi.Any(k => string.Equals(k.Baslik, girdi.Baslik?.Trim(),
        StringComparison.CurrentCultureIgnoreCase)))
    ModelState.AddModelError(nameof(KitapEkleVM.Baslik), "Bu başlıkta bir kitap zaten var.");

if (!ModelState.IsValid)
    return View(girdi);
```

Anahtar	Hata nerede görünür
`nameof(KitapEkleVM.Baslik)`	Başlık alanının altında
`string.Empty`	Yalnızca özette (`ModelOnly` ve `All`)

Sıra önemlidir: `AddModelError` satırı `IsValid` kontrolünden **önce** gelmelidir.

3.5. Hata varken neden yönlendirmiyoruz?

Hafta 4'teki PRG kuralı "POST'tan sonra yönlendir" diyordu. Kanonik kalıp ise geçersiz durumda **yönlendirmez**, `View(girdi)` döner. Çelişki değil, iki ayrı durum:

Durum	Ne yapılır	Neden
Geçerli, kayıt yapıldı	`RedirectToAction` (302)	F5 kaydı tekrarlamasın (Hafta 4)
Geçersiz, hiçbir şey kaydedilmedi	`View(girdi)` (200)	Tekrarlanacak bir yan etki yok; hatalar ve kullanıcının yazdıkları **bu yanıtta**

Geçersiz durumda yönlendirseydik `ModelState` kaybolurdu: yönlendirme **yeni bir istektir** ve `ModelState` istekle birlikte ölür (Hafta 4'te `ViewData` için gördüğümüz ömür kuralı). Kullanıcı boş bir formla ve hiçbir açıklama olmadan karşılaşırdı. Geçersiz formda F5'e basmak da zararsızdır: aynı POST yine geçersizdir ve yine hiçbir şey kaydedilmez.

---

4. İstemci Doğrulaması ve Sınırları

4.1. Nasıl çalışır?

Hafta 5'te `data-val-*` attribute'lerinin HTML'e kendiliğinden yazıldığını ve onları okuyan hiçbir şey olmadığını görmüştük. Okuyan iki kütüphanedir:

```
[Required(ErrorMessage = "{0} boş bırakılamaz.")]          ← modelde
        ↓ asp-for üretir
data-val-required="Başlık boş bırakılamaz."                  ← HTML'de
        ↓ jquery.validate.unobtrusive okur
required kuralı + mesaj                                      ← jQuery Validation'da
```

Unobtrusive kütüphanesi sayfa yüklenirken formu **bir kez** tarar ve kuralları kurar; **jQuery Validation** gönderimi yakalar, kurallar tutmazsa isteği **göndermez**.

Üç dosyanın yüklenme sırası zorunludur: önce **jQuery** (`_Layout.cshtml` sayfanın sonunda yükler), sonra **jQuery Validation**, en son **unobtrusive**. Görünümün `@section Scripts` bölümü `_Layout`'ta jQuery satırının **altında** basıldığı için (Hafta 3) partial'ı oraya koymak sırayı kendiliğinden doğru kurar. Betikleri bölümün dışına — örneğin formun hemen altına — koymak onları jQuery'den **önce** çalıştırır: konsolda `jQuery is not defined` hatası çıkar, sayfa yine açılır, doğrulama ise hiç kurulmaz.

4.2. Açmak: `_ValidationScriptsPartial`

Proje şablonunda hazır duran partial iki kütüphaneyi yükler (**TODO 6.5**):

```
@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

Ölçülen fark: boş form artık **sunucuya gitmez** (F12 → Network'te POST yok); mesajlar anında, sunucununkiyle aynı metinle görünür — çünkü ikisi de aynı attribute'ten gelir.

4.3. İstemci doğrulaması güvenlik değildir

İstemci doğrulaması açıkken üç atlatma denendi:

Deneme	Sonuç
F12 ile `data-val` attribute'lerini silip göndermek	**Yine engelledi** — kurallar sayfa açılırken kurulmuştu
Konsola `document.forms[0].submit()`	**Gönderildi** — tarayıcının ve jQuery'nin denetimi atlandı
JavaScript'i kapatıp göndermek	**Gönderildi**

Gönderilen iki durumda da sunucu **bütün** hataları döndürdü. Saldırgan formu hiç kullanmak zorunda da değildir: isteği doğrudan yazabilir. Bu yüzden:

> **İstemci doğrulaması kullanıcı deneyimidir; güvenlik sunucu doğrulamasıdır.**

4.4. Türkçe ondalık ayracı: istemci ile sunucu aynı dili konuşmalı

jQuery Validation sayıları İngilizce biçimde (`120.50`) bekler; Türkçe çalışan sunucu ise `120,50` bekler. Düzeltme yokken ölçülen:

Kullanıcı yazdı	İstemci	Sunucu (`tr-TR`)
`120,50`	**reddetti:** must be a number	— gönderilemedi
`120.50`	geçirdi	**12050** okudu → `[Range]` hatası
`12.50`	geçirdi	**1250** okudu ve **kaydetti**

Doğru değer gönderilemiyor, yanlış değer sessizce kaydediliyor. Örnek projede iki taraf birlikte düzeltildi:

- **Sunucu:** `Program.cs` 'te kültür `tr-TR` 'ye sabitlendi (`UseRequestLocalization`). Bu bir middleware'dir ve Hafta 2'den bildiğimiz kural geçerlidir: model bağlanmadan **önce** çalışmalıdır; bu yüzden pipeline'ın başında, `UseRouting` 'den önce durur. Sabitlenmezse kültür işletim sisteminden gelir ve aynı form başka makinede başka sayı bağlar.
- **İstemci:** `wwwroot/js/dogrulama-tr.js` , jQuery Validation'ın `number` ve `range` kurallarını Türkçe biçimi (`1.250,50`) kabul edecek şekilde değiştirir; `_ValidationScriptsPartial` onu iki kütüphaneden sonra yükler.

Düzeltmeden sonra `120,50` ve `1.250,50` iki tarafta da geçer, `12.50` istemcide durur. Eski davranışı görmek için: `/Dogrula?istemci=true&ham=true` .

4.5. Tek kaynak, iki taraf

İstemci ve sunucu kuralları ayrı ayrı yazılsaydı zamanla ayrışırlardı: biri güncellenir, öbürü unutulur. ASP.NET Core'da kural **bir kez**, modelde yazılır; iki taraf oradan beslenir:

Kural	Sunucuda	İstemcide
Hazır attribute (`[Required]`, `[Range]` ...)	✓	✓ — `data-val-*` olarak taşınır
Kendi `ValidationAttribute`'ünüz	✓	✗ — `IClientModelValidator` yazılmadıkça
`IValidatableObject`	✓	✗
`AddModelError` (iş kuralı)	✓	✗ — veri sunucuda

Tablonun sağ sütunu bir kez daha aynı şeyi söylüyor: istemci kuralların **bir kısmını** erken gösterir; tamamını yalnızca sunucu bilir. Mesaj metinleri de aynı kaynaktan geldiği için iki tarafta birebir aynıdır (ölçüldü: "Başlık boş bırakılamaz." hem tarayıcıda hem sunucu yanıtında).

---

5. Özel Doğrulama

5.1. Veri doğrulama mı, iş kuralı mı?

	Veri doğrulama	İş kuralı
Soru	Değer **kendi başına** geçerli mi?	İşlem **sistemin durumuna** göre yapılabilir mi?
Örnek	Başlık 2–100 karakter	Bu başlıkta kitap zaten var
Yeri	Model: attribute, `IValidatableObject`	Controller ya da servis: `AddModelError`
İstemcide çalışabilir mi?	Çoğu evet	Hayır — veri sunucuda

5.2. Kendi `ValidationAttribute`'ünüz

Hazır attribute'ler yetmediğinde aynı sınıftan türetirsiniz (**TODO 6.6**):

```
public class BuyukHarfleBaslarAttribute : ValidationAttribute
{
    public BuyukHarfleBaslarAttribute() : base("{0} büyük harfle başlamalıdır.") { }

    public override bool IsValid(object? value)
    {
        if (value is not string metin || metin.Length == 0)
            return true;                 // boşluğu [Required] yakalar
        return char.IsUpper(metin[0]);
    }
}
```

```
[BuyukHarfleBaslar]
[Display(Name = "Başlık")]
public string Baslik { get; set; } = "";
```

İki ayrıntı:

- **Boş değer bu attribute'ün işi değildir.** Her attribute tek bir şeyi denetler; boşluğu `[Required]` yakalar. Boşken `false` dönmek aynı alan için iki mesaj üretir.
- **Yalnızca sunucuda çalışır.** Ctrl+U: `Baslik` alanında `data-val-required` ve `data-val-length` var, bu attribute için **hiçbir** `data-val-*` yok. Ölçülen: küçük harfle başlayan başlık istemciden **geçti**, sunucu "Başlık büyük harfle başlamalıdır." ile geri çevirdi. İstemci tarafı için `IClientModelValidator` gerekir (`H06-detay.md` bölüm 2).

5.3. `IValidatableObject` — birden çok alana bakan kural

Bir attribute yalnızca **kendi** özelliğini görür. "Başlık ile yazar aynı olamaz" gibi iki alanı birlikte denetleyen kural modelin kendisine yazılır (**TODO 6.7**):

```
public class KitapEkleVM : IValidatableObject
{
    // … özellikler …

    public IEnumerable<ValidationResult> Validate(ValidationContext validationContext)
    {
        if (string.Equals((Baslik ?? "").Trim(), (Yazar ?? "").Trim(),
                StringComparison.CurrentCultureIgnoreCase))
            yield return new ValidationResult("Başlık ile yazar aynı olamaz.",
                [nameof(Baslik), nameof(Yazar)]);
    }
}
```

İkinci parametre hatanın **hangi alanlara** bağlanacağını söyler; mesaj iki alanın altında da görünür.

Ne zaman çalışır? `/Dogrula/Sira` sayfasında ölçüldü: iki alan da boş gönderildiğinde ikisi de `null`'dır — yani "aynıdır" — ama `Validate`'in mesajı **gelmedi**. Çatı `Validate`'i **yalnızca özellik düzeyindeki bütün kurallar geçtiyse** çağırır. Kitap formunda da aynısı ölçüldü: başlık ile yazar aynıyken fiyat da hatalıysa önce yalnızca fiyat hatası görünür; fiyat düzeltilince "aynı olamaz" hatası gelir.

5.4. Hangisi ne zaman?

Kural	Araç
Tek alan, hazır kalıp (boş, uzunluk, aralık, biçim)	Hazır attribute
Tek alan, kendi kalıbınız	Kendi `ValidationAttribute`'ünüz
Birden çok alan birlikte	`IValidatableObject`
Veri deposuna ya da başka bir servise bakmak gerekiyor	Controller'da `AddModelError`

5.5. Doğrulanmış veri, güvenli veri değildir

Doğrulama verinin **biçimini** denetler; her tehlikeyi değil. `Kitap <script>alert(1)</script>` başlığı bütün kurallardan geçer: 2–100 karakter, büyük harfle başlıyor, başka kitapta yok. Onu zararsız kılan doğrulama değil, Razor'ın çıktıyı **kodlamasıdır** (Hafta 3). Aynı ayrım ileride de geçerli:

Tehlike	Koruyan	Hafta
Biçimsiz ya da eksik veri	Doğrulama	6
Sayfaya enjekte edilen betik (XSS)	Razor'ın HTML kodlaması	3
Başka sitenin gönderdiği form (CSRF)	Anti-forgery token	5
Veritabanı sorgusuna enjekte edilen metin	EF Core'un parametreli sorguları	8
Yetkisi olmayan kullanıcının işlemi	Yetkilendirme	11

Doğrulama bu zincirin **ilk** halkasıdır; tek halkası değildir.

🧭 Sorun Çözme Köşesi — Form gönderildi, sonuç beklediğim gibi değil

Doğrulama sorunlarının çoğu "hangi katman konuştu?" sorusuyla çözülür. Önce F12 → **Network** sekmesine bakın:

```
Form gönderildi
├─ Ağda POST YOK
│   ├─ Alanın altında mesaj var ..... istemci doğrulaması durdurdu (bölüm 4)
│   └─ Alanın yanında tarayıcı balonu ..... tarayıcının yerleşik denetimi: type="email", maxlength (b
└─ Ağda POST VAR
    ├─ 302 ............................... geçerli sayıldı — olmaması gereken kayıt eklendiyse
    │                                      action IsValid'e bakıyor mu? (Klasik Tuzak)
    └─ 200 ............................... geçersiz sayıldı
        ├─ Mesaj İngilizce ................ bağlama hatası ya da ErrorMessage verilmemiş (bölüm 1.3,
        ├─ Mesaj Türkçe ................... sizin kuralınız çalıştı
        └─ Hiç mesaj yok .................. asp-validation-for eksik ya da hata formda olmayan
                                            bir alanda (Hata Köşesi)
```

Soru	Araç
İstek sunucuya gitti mi, hangi kodla döndü?	F12 → Network
Kurallar HTML'e taşındı mı? Betikler yüklü mü?	Ctrl+U: `data-val-*` ve `jquery.validate` satırları
Sunucu hangi alanda neyi beğenmedi?	`<partial name="_ModelStateTablosu" />` satırını formun altına geçici olarak ekleyin
İstemci doğrulaması neden kurulmadı?	F12 → Console: `jQuery is not defined`

🐛 Hata Köşesi — Hafta 6

Belirti	Gerçek sebep	Çözüm
`ModelState.IsValid` **hep** `false`, formda hata görünmüyor	Modelde formda **olmayan**, nullable olmayan bir referans özelliği var (gezinme özelliği gibi)	Özelliği ViewModel'den çıkarın, `?` ekleyin ya da `[ValidateNever]` yazın
`[Required] int Id` hiç hata vermiyor	`int` hiçbir zaman `null` olmaz; gelmediyse `0`'dır	Gerçekten zorunluysa `int?` yapın
Kendi mesajınız yerine **The value '' is invalid.**	Değer tipi boş geldi, bağlamada düştü	Tipi nullable yapıp `[Required]` ekleyin
`120,50` yazınca **must be a number**	İstemci İngilizce sayı biçimi bekliyor	`dogrulama-tr.js` + kültürü sabitlemek (bölüm 4.4)
`12.50` yazılan fiyat **1250** olarak kaydedildi	Sunucu `tr-TR`'de noktayı binlik ayracı okur	Aynı düzeltme — istemci `12.50`'yi artık reddeder
İstemci doğrulaması **hiç çalışmıyor**	Sayfada `_ValidationScriptsPartial` yok	`@section Scripts { <partial name="_ValidationScriptsPartial" /> }`
`asp-validation-summary="ModelOnly"` **boş**	Bütün hatalar bir alana bağlı	Normaldir; alana bağlı olmayan hata için `AddModelError(string.Empty, …)`
`AddModelError` hatası görünmüyor	Satır `IsValid` kontrolünden **sonra** yazılmış	Kontrolden önceye alın
`IValidatableObject` kuralı çalışmıyor gibi	Başka bir alanda hata var; `Validate` henüz çağrılmadı	Önce diğer hataları düzeltin — tasarım böyle

🕳️ Klasik Tuzak — Kuralı yazıp sonucuna bakmamak

İki yarısı var.

1. Yalnızca istemciye güvenmek. "Form zaten boş gönderilemiyor" — tarayıcıda. Konsoldan tek satırla (`document.forms[0].submit()`) ya da formu hiç kullanmadan gönderilen istek sunucuya ulaşır.

2. `IsValid`'i kontrol etmemek. Kurallar yazıldı, çatı onları çalıştırdı, hatalar `ModelState`'e yazıldı — ama action okumadı:

```
[HttpPost]
public IActionResult EkleTH(KitapEkleVM girdi)
{
    KitapDeposu.Ekle(/* … */);     // ❌ IsValid kontrolü yok
    return RedirectToAction(nameof(Index));
}
```

Ölçülen: boş form **302** ile kabul edildi ve listeye başlığı **boş**, fiyatı **0 TL** bir kitap eklendi. Doğrulama attribute'leri bir şeyi **engellemez**; yalnızca **raporlar**. Engelleyen `if (!ModelState.IsValid)` satırıdır.

📖 Terimler Sözlüğü

Terim	Ne demek
Doğrulama (validation)	Gelen verinin kurallara uyup uymadığını işlemden önce denetlemek
`ModelState`	Bağlama ve doğrulama sonuçlarını alan alan tutan sözlük
`ModelState.IsValid`	Hiçbir alanda hata yoksa `true`
DataAnnotations	Kuralları özelliklere attribute olarak yazmayı sağlayan ad alanı
`ErrorMessage`	Attribute'ün mesajı; `{0}` alan adı, `{1}` ve `{2}` parametreler
Örtük doğrulama	Attribute yazılmadan gelen denetim: nullable olmayan referans tipi, bağlama
`asp-validation-for`	Bir alanın hatasını gösteren Tag Helper
`asp-validation-summary`	Hataları liste hâlinde gösteren Tag Helper (`All` / `ModelOnly`)
`AddModelError`	`ModelState`'e elle hata eklemek; anahtar boşsa model düzeyinde
İstemci doğrulaması	`data-val-*` + jQuery Validation ile tarayıcıda yapılan denetim
Unobtrusive	`data-val-*` attribute'lerini okuyup jQuery Validation kurallarına çeviren kütüphane
`ValidationAttribute`	Bütün doğrulama attribute'lerinin türediği sınıf
`IValidatableObject`	Birden çok alanı birlikte denetleyen model düzeyi kural
İş kuralı	Sistemin durumuna bağlı kural; controller ya da serviste denetlenir
Ham metin (`AttemptedValue`)	İstemcinin gönderdiği metin; çevrilemese de saklanır ve forma geri yazılır
Tarayıcının yerleşik denetimi	`type="email"` , `maxlength` gibi HTML attribute'lerini tarayıcının kendisinin denetlemesi

---

✅ Kontrol Soruları

A. Doğrulama nerede çalışır

1. Bir POST action'ının ilk satırı çalıştığında Model Binding ve doğrulama hangi durumdadır?
2. Geçersiz bir form gönderildiğinde yanıt neden 400 değil de 200'dür?
3. `int Sayi` alanı boş gönderildi; hatayı kim yazdı, mesajı neden İngilizce?
4. Bir alan formda **hiç yoksa** ile **boş** gönderilirse ne fark eder?
5. Üç doğrulama katmanını sayın. Hangisi atlatılamaz?

B. DataAnnotations

6. `[StringLength(100, MinimumLength = 2, ErrorMessage = "…")]` mesajında `{1}` ve `{2}` neyle dolar?
7. `decimal Fiyat` alanına `[Required(ErrorMessage = "…")]` yazdınız; boş gönderince neden sizin mesajınız gelmiyor?
8. `[DataType(DataType.EmailAddress)]` ile `[EmailAddress]` arasındaki fark nedir?
9. `[RegularExpression]`'a neden her zaman `ErrorMessage` verilmelidir?
10. Parola tekrarı alanı için hangi attribute kullanılır? Parametresine neden `nameof` yazılır?

C. ModelState ve gösterim

11. Kanonik POST action'ını yazın: geçersizse ne döner, geçerliyse ne döner?
12. `Fiyat` alanına `abc` yazılıp gönderildi; form geri geldiğinde kutuda ne yazar? Neden?
13. `asp-validation-summary="All"` ile `"ModelOnly"` arasındaki fark nedir?
14. `AddModelError(nameof(Baslik), …)` ile `AddModelError(string.Empty, …)` hataları nerede görünür?
15. `ModelState.IsValid` hep `false` ama ekranda hata yok. İlk neye bakarsınız?

D. İstemci doğrulaması

16. `data-val-*` attribute'lerini hangi kütüphane okur? Ne zaman okur?
17. İstemci doğrulamasını açmak için görünüme ne eklenir?
18. F12 ile `data-val` attribute'lerini silmek doğrulamayı neden atlatmadı?
19. İstemci doğrulaması varken sunucu doğrulamasına neden hâlâ ihtiyaç var?
20. `12.50` yazılan fiyatın 1250 olarak kaydedilmesinin sebebi ne? Örnek proje bunu nasıl önlüyor?

E. Özel doğrulama

21. "Bu başlıkta kitap zaten var" kuralı neden bir attribute ile yazılmaz?
22. Kendi `ValidationAttribute`'ünüzde değer boşsa neden `true` dönmelisiniz?
23. Kendi attribute'ünüz istemci tarafında neden çalışmaz?
24. `IValidatableObject.Validate` hangi koşulda çağrılmaz?
25. Başlık ile yazarın aynı olamayacağı kuralını hangi araçla yazarsınız? Hata hangi alanların altında görünür?

📑 Kaynakça

> Her kaynağın yanında **bu ders notunun hangi bölümünü** dayandırdığı yazılıdır. Bağlantıların sonundaki `?view=aspnetcore-10.0` **kasıtlıdır**: sürüm sabitlenmezse arama motoru sizi eski sürüme düşürür.

Zorunlu okuma

Kaynak	Dayandırdığı bölüm

Model validation in ASP.NET Core MVC and Razor Pages — https://learn.microsoft.com/aspnet/core/mvc/models/validation?view=aspnetcore-10.0	Bölüm 1–5 — akış, örtük `[Required]`, nullable kuralı, istemci doğrulaması, özel attribute, `IValidatableObject`
The Validation Tag Helpers — https://learn.microsoft.com/aspnet/core/mvc/views/working-with-forms?view=aspnetcore-10.0#the-validation-tag-helpers	Bölüm 3.3 — `asp-validation-for`, `asp-validation-summary`
System.ComponentModel.DataAnnotations — https://learn.microsoft.com/dotnet/api/system.componentmodel.dataannotations	Bölüm 2 — hazır attribute'lerin tam listesi

Derinleştirme

Kaynak	Dayandırdığı bölüm
Model Binding in ASP.NET Core — https://learn.microsoft.com/aspnet/core/mvc/models/model-binding?view=aspnetcore-10.0	Bölüm 1.3, 2.3 — bağlama hataları
Globalization and localization in ASP.NET Core — https://learn.microsoft.com/aspnet/core/fundamentals/localization?view=aspnetcore-10.0	Bölüm 4.4 — `UseRequestLocalization`
jQuery Validation Plugin — https://jqueryvalidation.org/documentation/	Bölüm 4.1, 4.4 — `number` ve `range` kuralları
OWASP Input Validation Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html	Bölüm 4.3 — istemci doğrulamasının güvenlik sayılmaması

📚 Daha Fazlası

`H06-detay.md` dosyasında (**sınav kapsamı dışı**):

- Doğrulamanın içeriden işleyişi: `ValidationVisitor`, en fazla 200 hata sınırı, `[ValidateNever]`
- Kendi attribute'ünüzü istemcide çalıştırmak: `IClientModelValidator`
- Bağlama mesajlarını Türkçeleştirmek: `ModelBindingMessageProvider` (`H05-detay.md` bölüm 6'nın devamı)
- `[Remote]`: istemciden sunucuya soran doğrulama
- FluentValidation: kuralları modelin dışına taşımak
- Kültür ve sayılar: `UseRequestLocalization`'ın içi, `type="number"` seçeneği
- Tarayıcının yerleşik denetimi ile jQuery Validation'ın ilişkisi: `novalidate`, `formnovalidate`

➡️ Haftaya Hazırlık

Bu hafta her isteği **tek başına** doğruladık. Ama HTTP her isteği unutur: kullanıcı sayfayı yenilediğinde ya da başka bir sayfaya geçtiğinde, sunucu onun **kim** olduğunu ve sepetinde **ne** olduğunu nereden bilecek?

Hafta 7'de bunun iki cevabını kuracağız: **cookie** ve **session**.

Düşünerek gelin: Sepetinize eklediğiniz ürün ikinci sayfaya geçtiğinizde nerede duruyor — sunucuda mı, tarayıcıda mı? Bu haftanın ilkesi — *güvenme, doğrula* — tarayıcıda duran bir veri için ne söyler?

> 🏠 Ev göreviniz bu haftanın sınav provasıdır: kendi formunuza en az beş kural, kanonik action ve **sunucunun gerçekten denetlediğinin** kanıtı.